

One-Day Development Plan

Student Expense Tracker with Analytics | Team 17 | SWE 4637

Golden Rules Before You Start

- Build the smallest working version first (MVP). Add extra polish only if time remains.
- Agree on the API contract (URLs, request/response fields) in the first 30 minutes, before anyone writes real code. This is the #1 way to avoid wasted hours.
- Each person works on their own files/folders to avoid Git conflicts. Push and pull often (every 30–45 min).
- Test each small piece as you build it. Do not wait until the end to test everything together.
- If a feature is taking too long, cut it. Follow the 'Cut List' at the end of this plan.

Team Roles

Person	Role	Owns
Person A (Ahnaf)	Backend & Database Lead	FastAPI setup, PostgreSQL + SQLAlchemy models, CRUD APIs for expenses, categories, budgets
Person B (Sameur)	Frontend Lead	ReactJS + Tailwind setup, pages, routing, forms, expense list/UI, dashboard layout
Person C (Irfan)	Auth, Analytics & Integration Lead	JWT login/register, Recharts analytics, CSV export, connecting frontend to backend, end-to-end testing

Swap names/roles if someone is stronger in a different area — the important part is that each area has ONE clear owner.

Must-Have (MVP) vs Nice-to-Have

MUST-HAVE (build first, in order)	• Register / Login (JWT) • Add, edit, delete an expense (with category) • View list of expenses • Add a monthly budget per category + simple overspend warning • One working chart (e.g. pie chart of spending by category) • Basic responsive layout (does not need to be fancy)
NICE-TO-HAVE (only if time remains)	• Bar chart + line chart (in addition to pie chart) • CSV export • Filtering / searching expenses • Polished UI animations, dark mode, etc.

Hour-by-Hour Timeline (assumes a ~11 hour day, 9:00 AM start)

Adjust clock times to whenever you actually start — what matters is the ORDER and the relative length of each block.

9:00 – 9:30 AM Sprint 0: Kickoff & Setup (30 min, ALL TOGETHER)		
Goal: Everyone agrees on the plan before splitting up.		
Person A	Person B	Person C
• Create GitHub repo, invite teammates • Create backend/ and frontend/ folders • Draft the API contract together (endpoints + fields)	• Confirm API contract • List all pages/screens needed • Agree on Tailwind color/theme basics	• Confirm API contract • Confirm JWT flow (what token looks like, header format) • Set up shared doc/Trello for tracking tasks
9:30 – 11:30 AM Sprint 1: Core Foundations		
Goal: Get the skeleton of backend and frontend running and talking to a database.		
Person A	Person B	Person C
• Set up FastAPI project structure • Set up PostgreSQL + SQLAlchemy connection • Create models: users, categories, expenses, budgets • Set up Alembic migrations, run first migration	• Set up React app (Vite) + Tailwind • Set up React Router with page stubs: Login, Register, Dashboard, Expenses • Build basic layout/navbar	• Build register & login endpoints (JWT) • Build password hashing (bcrypt/passlib) • Write a simple Postman/Thunder Client collection to test endpoints
11:30 AM – 1:30 PM Sprint 2: Core CRUD Features		
Goal: The main features start working end-to-end for expenses.		
Person A	Person B	Person C
• Build CRUD endpoints: /expenses (create, read, update, delete) • Build CRUD endpoints: /categories • Add basic input validation (Pydantic schemas)	• Build Login/Register forms, connect to backend auth • Build 'Add Expense' form (amount, category, description, date) • Build Expense List view (table/cards)	• Build JWT middleware to protect routes • Connect frontend axios/fetch calls to auth endpoints • Store JWT in frontend (context or localStorage) and attach to requests

1:30 – 2:15 PM Lunch / Short Break		
Goal: Rest. Quickly re-sync on blockers before continuing.		
Person A	Person B	Person C
• Push your code before break	• Push your code before break	• Push your code before break
2:15 – 4:15 PM Sprint 3: Budgets, Alerts & Feature Completion		
Goal: Finish the remaining must-have backend logic and connect it to the UI.		
Person A	Person B	Person C
• Build CRUD endpoints: /budgets • Add overspend-check logic (compare expenses vs monthly_limit) • Add filtering: expenses by category/date range	• Build Expense edit/delete actions in the UI • Build 'Set Budget' form + budget list view • Show overspend warning banner/badge in UI	• Connect Expense List/Form to backend CRUD • Connect Budget form to backend • Test full flow: login → add expense → set budget → see warning
4:15 – 6:00 PM Sprint 4: Analytics Dashboard		
Goal: Add the charts — this is the 'Analytics' part of the project.		
Person A	Person B	Person C
• Build /analytics endpoint(s): totals by category, totals by month • Optimize queries if slow • Help debug integration issues	• Build Dashboard page layout • Integrate Recharts: Pie chart (spend by category) — MUST HAVE • If time: Bar chart (by category) and Line chart (by month)	• Connect Dashboard to /analytics endpoint • Build CSV export button (nice-to-have) calling backend export endpoint • Cross-browser / responsive check
6:00 – 7:15 PM Sprint 5: Integration Testing & Bug Fixing (ALL TOGETHER)		
Goal: Everyone stops building new features. Test the whole app together as a real user would.		
Person A	Person B	Person C
• Fix backend bugs found during testing • Double-check error handling (bad input, expired token)	• Fix UI bugs, loading states, empty states • Make sure forms show validation errors clearly	• Run through every user flow start to finish • Keep a running bug list, assign fixes
7:15 – 8:00 PM Sprint 6: Final Polish & Demo Prep		
Goal: Make it presentable.		

Person A	Person B	Person C
• Clean up unused code / print statements • Make sure DB seed/demo data exists	• Quick visual polish (spacing, colors, consistency) • Make sure app doesn't crash on empty states	• Prepare a 3–5 min demo script • Take screenshots for the report/slides

The Cut List — What to Drop if You're Running Out of Time

Cut items in this order (top of the list goes first). Never cut anything from the MVP list unless truly desperate.

- 1 CSV export
- 2 Bar chart and line chart (keep only the pie chart)
- 3 Filtering/searching expenses
- 4 Edit budget (keep only create budget)
- 5 UI polish (animations, icons, extra styling)
- 6 Category management UI (hardcode 5–6 default categories instead)

Team Sync Points

- Every 2 hours: quick 5-minute stand-up — what's done, what's blocked, what's next.
- Push to Git after every completed task, not just at the end of a sprint.
- If Person A finishes backend endpoints early, they should help Person C with integration or Person B with data-related bugs.
- Communicate immediately if an endpoint's fields change — this breaks the other side silently otherwise.